

第 33 章 RHCSA 综合任务与证据矩阵

把跨用户、网络、安全、存储、调度与容器的要求，组织成可恢复、可验证、能够通过重启门的完整终态。

对象模型风险排序分层验收诊断恢复综合任务

大字号阅读版

本章阅读导航

先抓住一条主线：综合任务不是命令清单，而是把初始状态经过受控变更推进到一组互相一致、能够被证据逐项证明的终态。

- 01 拆解终态把题目改写为对象、参数、限制和证据
- 02 建立依赖识别硬依赖、共享对象和冲突域
- 03 调查基线确认环境事实，不猜接口、设备和已有状态
- 04 风险分批低风险先闭环，高风险操作通过证据门
- 05 分层验收分别证明静态、当前、功能和持久状态
- 06 重启与恢复共享对象回归，重启后按矩阵重新验收

专题地图

- *知识专题* 从题目原文提取评分对象与可验证终态
- *知识专题* 构造依赖图、共享对象和冲突域
- *知识专题* 风险排序、证据门、恢复点与执行批次
- *操作专题* 建立整机调查基线和任务账本
- *操作专题* 建立四层证据矩阵
- *操作专题* 用局部闭环和回归集合推进任务
- *诊断专题* 卡题、误操作和交叉影响的证据式恢复
- *经典任务* 五类领域任务与一次模拟考试级终检

阅读时持续回答

1. 题目真正评分的是哪个对象和状态？
2. 哪些参数来自题目，哪些环境事实必须查询？
3. 哪些任务存在硬依赖或共享对象？
4. 下一步操作的风险等级和停止条件是什么？
5. 当前证据能证明哪一层，又不能证明哪一层？
6. 修改后最小功能验证是什么？
7. 重启前还缺哪些静态检查与恢复入口？
8. 重启后应从哪张矩阵逐项复查？

方法提醒：先从原题建立评分行，再按依赖和风险推进；每次变更都要留下能够说明证明范围的证据。

第 33 章 · 正文

RHCSA 综合任务最容易制造一种错觉：题目很多、命令很多，于是只要按记忆把命令逐条敲完，整台系统就会自然到达正确终态。真实情况恰好相反。用户、网络、仓库、服务、端口、SELinux、存储、挂载、计

划任务和容器分别属于不同对象层；它们又通过用户身份、路径、设备、端口、名称解析和启动生命周期互相连接。一个局部命令成功，可能只说明当前动作没有报错，并不能证明最终功能正确，更不能证明系统重启后仍然成立。

本章不重讲前 32 章的子系统基础，而是把已经建立的对象组合成一套稳定的综合执行方法。核心不是背一份固定考试顺序，而是把每道题转换成可验证终态，识别依赖与共享对象，控制高风险操作，并用证据矩阵逐项关闭缺口。各技术机制在需要时回链对应章节；RHCE 自动化实现不在本章展开。

概念

综合任务 是多个系统对象和限制条件共同构成的终态问题。它的评分对象不是使用过哪些命令，而是用户、路径、连接、服务、设备、挂载、调度或容器最终是否满足题目。命令只是查询或改变对象的接口，因此“命令执行成功”不能替代对终态的验收。

概念

依赖关系与共享对象 决定执行顺序和回归范围。硬依赖不成立时，后续功能无法建立或验证；共享对象则会被多个任务共同使用，例如一个用户同时关联 ACL、cron 和 rootless 容器。修改共享对象后，只回归相关任务，但不能假设先前通过的状态仍然成立。

概念

风险排序与证据门 用于控制不可逆或高影响操作。网络切换、分区、文件系统签名、fstab 和重启都可能扩大故障面；进入这些步骤前，必须确认对象身份、保持条件、停止条件和恢复入口。证据不足时应标记为 blocked，而不是用 force、删除重建或全局放宽安全策略换取短暂成功。

概念

调查基线 是变更前对关键对象身份和状态的最小快照。它不是把所有查询命令运行一遍，而是只收集会改变决策的字段：实际 profile、设备签名、已有挂载、用户关系、活动 zone、配置来源和消费者身份。没有基线，就无法判断变化是否由本次操作引起，也难以进行最小恢复。

概念

四层证据 分别回答不同问题：静态证据证明声明或语法可解析；当前证据证明对象此刻已经加载或生效；功能证据从真实消费者角度证明可用；持久或重启证据证明生命周期跨越后仍成立。题目要求哪一层，就必须给出能够证明那一层的证据，不能用 active、文件存在或退出码互相替代。

概念

验收矩阵 把原始要求、目标对象、依赖、风险、四层证据和剩余缺口放在同一记录中。它的价值不是表格本身，而是让每个 PASS 都能追溯到具体证据，并让 pending、blocked 与 deferred 明确说明仍缺什么。

概念

恢复路径 是失败前预先保留的返回入口和判断顺序。文本配置可以依靠保留属性与上下文的副本恢复；网络需要控制台或第二会话；启动故障需要明确进入救援路径；存储写入则不能靠复制配置文件撤销。诊断应从最后一条可信证据开始，选择最有区分度的下一步，而不是反复执行失败命令。

操作语义

本章不以某一个命令为中心。下面的入口把“任务拆解、对象调查、分层验收、高风险证据门、最终重启和错误恢复”组织成可重复的操作接口；详细命令仍在对应专题与经典任务中按对象展开。

任务拆解

SYNOPSIS

原始要求 → 对象 → 精确参数 → 限制 → 必需证据

把自然语言题面转换成可逐项判定的评分行。

对象

用户、路径、连接、unit、端口、设备、卷、挂载、调度或容器。

参数

题目明确给出的 UID、地址、端口、容量、路径和时间。

限制

不得删除数据、不得降低 SELinux、必须保留已有对象等。

证据

写明需要静态、当前、功能、持久或重启中的哪些层。

基线查询组

SYNOPSIS

getent | id | stat | getfacl | nmcli | lsblk | findmnt | systemctl

确认对象身份、配置来源和初始状态；只查询会改变下一步决策的字段。

身份与权限

getent、id、namei、getfacl、sudo -l。

网络与服务

nmcli、ip、getent hosts、systemctl、ss。

存储

lsblk、blkid、pvs/vgs/lvs、findmnt、swapon。

容器与调度

podman、systemctl --user、loginctl、journalctl。

执行顺序规划

SYNOPSIS

硬依赖 → 低风险前置 → 局部闭环 → 共享对象回归

依赖决定能否继续，风险决定需要多少前置证据，共享对象决定修改后的回归集合。

硬依赖

网络和名称解析不成立时，仓库、NFS 与镜像访问无法可靠验收。

低风险前置

先完成只读调查和不会破坏现状的对象创建。

局部闭环

一次只推进一个状态层，修改后立即做最小验证。

回归集合

用户、路径、端口、设备或 profile 变化后复查所有消费者。

分层验收

SYNOPSIS

```
VERIFY {static | current | functional | persistent}
```

每条证据只承担一个证明目标，并明确不能证明的相邻状态。

static

配置、语法、声明和稳定标识。

current

此刻加载、运行、监听、挂载或激活。

functional

目标用户、客户端或数据消费者实际完成动作。

persistent

重新登录、reload、退出会话或重启后仍成立。

高风险证据门

SYNOPSIS

```
IDENTITY + KEEP + STOP + RECOVERY → WRITE
```

在网络切换、分区、mkfs、fstab 和重启前，先证明对象身份与保持条件，并准备停止点和恢复入口。

IDENTITY

确认真实接口、profile、设备、签名、挂载和使用者。

KEEP

明确不得删除、覆盖或降低保护的状态。

STOP

证据不完整或输出矛盾时停止写入。

RECOVERY

准备第二会话、控制台、配置副本或启动恢复路径。

重启与恢复决策

SYNOPSIS

```
静态检查通过 + 当前功能闭环 + 恢复入口可用 → reboot
```

重启用于验证持久链，不用于猜故障；失败后从最后可信层开始诊断。

重启前

检查网络自动连接、fstab、关键服务、用户 unit 和剩余 blocked 项。

重启后

按原始题目和验收矩阵逐项复查，不凭命令历史判断。

失败恢复

症状 → 当前证据 → 假设 → 高区分度证据 → 最小修复 → 再验证。

[知识专题] 从题目原文提取评分对象与可验证终态

综合题的第一步不是打开终端，而是把自然语言要求转换成一组可以逐项判定的状态。题目常使用“配置”“确保”“允许”“创建”“运行”等动词；这些动词没有直接告诉你应该用哪条命令，也没有说明一条命令成功后还需要哪些证据。稳定切入点是先找对象，再补齐参数、限制和生命周期。

① [知识点] 模糊动词必须改写为“对象 + 参数 + 终态”

例如“配置 Web 服务”至少可能包含：软件存在、配置语法正确、服务当前运行、开机自动启动、监听指定端口、SELinux 允许、firewalld 放行、内容可读取、远端客户端能访问。题面没有要求的层不要擅自扩展，但题面明确要求的层不能被一句“服务已启动”替代。

建议把每项要求改写为：

原始要求

- 目标对象
- 精确参数
- 当前终态
- 持久终态
- 功能验收
- 禁止事项或保持条件

② [知识点] 一道题可能包含多个独立评分行

“创建用户 deploy，使其属于 opsadmins，并能在 /srv/project 中协作”至少涉及三个对象：账号、附加组关系、目录访问。若再要求新文件自动继承组和审计用户只读，则还涉及 setgid 与默认 ACL。把所有内容写在一行会导致某个子状态失败时无法定位缺口。

任务账本应把它们拆成独立行，同时保留原题编号，避免最后检查时只看到“用户题已做”。

③ [知识点] 正向要求、限制条件和保持现状同样重要

下面三类内容都必须进入终态模型：

- 正向状态：服务监听 8082/tcp；
- 限制条件：SELinux 保持 Enforcing；
- 保持条件：不得删除 /srv/project 中现有文件。

只记录正向状态会诱导出高风险捷径，例如清空目录重建、删除现有网络连接、关闭 SELinux 或重新格式化不明设备。保持条件应被视为验收项，而不是旁注。

④ [知识点] 命令不是评分对象，也不能代替证据

执行过 `systemctl enable --now httpd` 只能证明命令被调用；它不能单独证明配置语法正确、进程没有立即退出、监听端口正确、远端访问成功。命令历史最多是变更记录，真正的评分证据来自对象状态。

⑤ [知识点] 将题目参数与环境事实分开

题目给出的 IP、端口、大小和路径属于目标参数；接口名、现有连接 profile、设备签名、UUID 和当前 zone 属于环境事实，必须查询，不能猜测。即使训练题给出 `/dev/vdb`，正式操作前仍应通过 `lsblk`、`blkid` 和挂载关系确认它没有被其他对象使用。

[Cheatsheet] 先写对象，再写参数；把一道大题拆成独立评分行；限制条件进入矩阵；命令历史不是终态证据；题目参数可以照抄，环境事实必须查询。

[知识专题] 构造依赖图、共享对象和冲突域

综合任务不是一条固定线性流水线。有些要求存在硬依赖，有些只是为了降低返工而适合提前完成，还有一些彼此独立，可以按风险和耗时自由安排。依赖图的目标不是画得复杂，而是避免在前置条件尚未成立时误判后续故障。

① [知识点] 区分硬依赖、顺序优化和独立任务

硬依赖表示前置对象不成立，后续对象无法正确完成。例如仓库不可用时不能可靠安装缺失软件；NFS 名称无法解析时 `autofs` 功能验收无法完成。

顺序优化表示后续任务并非绝对不能做，但先完成前置工作可以减少回归。例如先完成用户和组，再设置目录 ACL，可以避免后续修改所有权时重新检查访问。

独立任务不应被人为串成一条长链。若本地日志查询与未使用该日志的 LVM 创建没有共享对象，两者可以独立闭环。

② [知识点] 共享对象决定回归范围

典型共享对象包括：

- 用户与组：ACL、sudo、计划任务、rootless 容器；
- 路径：挂载、SELinux 标签、服务 DocumentRoot、容器卷；
- 端口：应用配置、监听、SELinux 端口类型、firewalld；
- 设备：分区、PV、VG、文件系统、Swap、fstab；
- NetworkManager profile：地址、路由、DNS、仓库、NFS、远程访问；
- 时间：日志时间线、cron 触发、证据关联。

修改共享对象后不需要把整台主机从头测试，但必须重新执行与它相关的回归集合。

③ [知识点] 跨层服务访问链必须逐层证明

一个非标准端口的 HTTP 服务通常经过：

配置文件

→ 配置语法

→ 服务进程

→ 监听套接字

→ SELinux 文件与端口策略

→ firewalld

→ 本机请求

→ 远端请求

任何一层失败都可能表现为“访问不了”。直接反复重启服务无法区分这些原因。

④ [知识点] 存储路径会覆盖已有目录视图

把文件系统挂载到一个已有内容的目录后，原目录内容不会被删除，但会被挂载对象遮蔽。若服务或容器已经使用该路径，挂载顺序和数据迁移必须先明确。综合任务中创建挂载点前，应检查 `findmnt`、目录内容和使用者；不能只确认目录是否存在。

⑤ [知识点] 用户级 systemd 依赖登录生命周期

rootless 容器、用户 Quadlet 和 `systemctl --user` 不只依赖容器定义，还依赖目标用户、用户 manager、运行时目录和 linger。用 root 上下文运行 `podman` 成功不能证明目标用户的容器会在退出登录后保持。

[Cheatsheet] 硬依赖决定能不能做；顺序优化决定是否容易返工；共享对象决定回归范围；服务访问按配置、进程、端口、安全和客户端逐层证明。

[知识专题] 风险排序、证据门、恢复点与执行批次

综合任务的稳定顺序通常不是按题号，也不是把最熟悉的题先做完。更可靠的方法是先完成低风险、强前置和能改善调查能力的对象；进入网络切换、分区、格式化、fstab 和重启等高影响动作前设置证据门。

① [知识点] 风险按影响面和可恢复性分级

可使用以下简化等级：

等级	典型操作	主要策略
R0 只读	<code>getent</code> 、 <code>lsblk</code> 、 <code>nmcli show</code>	尽早执行并保存基线
R1 低风险	创建用户、写新 repo 文件	操作后立即局部验证
R2 可中断服务	修改服务配置、重启 daemon	先做语法检查与恢复副本
R3 连接或共享影响	切换网络 profile、修改防火墙	保留控制台或第二会话
R4 数据/启动关键	分区、mkfs、fstab、重启	必须通过证据门和恢复入口

风险等级不是禁止操作，而是决定需要多少前置证据和回退准备。

② [知识点] 高风险写操作需要证据门

分区前的最低证据门包括：目标设备身份、容量、当前挂载、文件系统签名、是否属于 LVM/Swap。fstab 写入后的最低证据门包括：语法验证、目标存在、手工挂载成功、重复项检查。未通过证据门时应标记为 blocked，而不是继续尝试破坏性参数。

③ [知识点] 网络修改必须保留可达性恢复路径

通过 SSH 修改当前连接时，应优先保留控制台或第二会话，并避免先删除正在工作的 profile。可以创建或修改候选 profile，核对全部属性后再激活；新连接验证成功后，再处理旧 profile 的自动连接优先级。

④ [知识点] 配置副本不是万能回滚

对文本配置，保留带权限和上下文的副本有助于恢复；对 LVM、文件系统和分区，复制文本配置不能恢复数据结构。高风险存储操作的真正安全边界是确认对象身份、避免误写，以及在写入前停止操作。

⑤ [知识点] 重启是最终验收门，不是首轮排错工具

重启会同时改变服务、挂载、网络、用户 manager 和容器状态，使故障面扩大。只有启动关键配置已经通过静态检查，当前功能已闭环，恢复入口可用时，才进入重启。重启后按既有矩阵复查，不随机重做全部任务。

[Cheatsheet] 先只读，再低风险；网络保留第二入口；分区和 fstab 有证据门；文本副本不能撤销存储写入；重启用于证明持久性，不用于猜故障。

[操作专题] 建立整机调查基线和任务账本

调查基线的目的不是收集所有可能信息，而是为当前题目提供对象身份、初始状态和冲突证据。命令应围绕题目涉及的对象选择，输出应保存到可回看的工作记录中。下面的宽基线是候选集合，不要求每场考试机械执行全部命令。

① [查询] 建立主机、时间与启动基线

作用对象：主机身份、当前启动、时间可信度和默认 target。

```
hostnamectl
uname -r
uptime
who -b
timedatectl
systemctl get-default
systemctl --failed
journalctl -b -p warning..alert --no-pager
```

`systemctl --failed` 只能提示失败 unit，不能证明所有业务正常。日志时间线若依赖错误时钟，也不能直接用于判断先后顺序。

② [查询] 建立身份、权限和路径基线

```
getent passwd deploy auditor svcweb
getent group opsadmins
id deploy
sudo -l -U deploy
stat -c '%n %U:%G %a %A' /srv/project
namei -om /srv/project
getfacl -p /srv/project
findmnt -T /srv/project
```

`namei -om` 用于逐级观察路径权限；`findmnt -T` 用于确认路径实际位于哪个挂载对象。只看末级目录权限可能遗漏父目录不可遍历或被挂载覆盖。

③ [查询] 建立网络、仓库、服务与安全基线

```
nmcli -f NAME,UUID,TYPE,DEVICE,AUTOCONNECT connection show
nmcli -f GENERAL,IP4,IP6 device show
ip -brief address
ip route
getent hosts repo.lab.example.com nfs.lab.example.com

dnf repolist -v
systemctl status httpd --no-pager
ss -lntup
firewall-cmd --get-active-zones
firewall-cmd --list-all
firewall-cmd --permanent --list-all
getenforce
semanage port -l | grep -E '^http_port_t'
```

firewalld 的 runtime 与 permanent 是两个状态面；`ss` 证明本机监听，不证明防火墙和远端路径。

④ [查询] 建立存储与挂载基线

```
lsblk -e7 -o NAME,PATH,SIZE,TYPE,FSTYPE,FSVER,LABEL,UUID,MOUNTPOINTS
blkid
pvs
vgs
lvs -a -o lv_name,vg_name,lv_size,lv_attr,devices
findmnt --real
swapon --show
cat /etc/fstab
```

目标设备必须同时结合 `lsblk`、`blkid`、LVM 归属和挂载关系判断。设备名存在不代表它空闲。

⑤ [查询] 建立调度、远程文件系统和容器基线

```
systemctl status autofs crond chronyd --no-pager
automount -m
chronyc sources -v
chronyc tracking
ls -l /etc/cron.d

loginctl show-user svcweb -p Linger -p State
# 以下命令必须在 svcweb 的真实用户会话中执行
podman info
podman ps -a
systemctl --user --failed
```

root 运行的 `podman ps` 与 `svcweb` 的容器存储完全不同。调查 `rootless` 对象时必须明确用户上下文。

⑥ [操作] 建立任务账本

建议维护如下字段：

U-01 · 创建 `deploy`

对象：用户；初始状态：不存在；依赖：无；风险：R1。

下一动作：创建账号；缺失证据：身份记录与新登录会话；状态：`planned`。

W-03 · 远端访问 8082

对象：完整服务访问链；初始状态：未知；依赖：网络、`httpd`、`SELinux` 与 `firewalld`；风险：R2。

下一动作：先确认监听；缺失证据：远端功能；状态：`blocked`。

状态可以使用 `planned`、`in_progress`、`verified`、`blocked`、`deferred`。不要把“执行过”当成 `verified`。

[Cheatsheet] 基线只围绕题目对象；身份、路径、网络、存储和用户上下文分开记录；任务账本记录缺失证据而不是命令次数。

[操作专题] 建立静态、当前、功能和持久证据矩阵

证据矩阵把“我做过什么”改写为“系统现在能够证明什么”。每项要求不必机械填满四层，但必须覆盖题目要求的生命周期。证据命令应尽量短、可重复、结果具有唯一判断边界。

① [验证] 静态配置证据

静态证据回答“声明是否正确”。典型入口包括：

```
visudo -cf /etc/sudoers
httpd -t
findmnt --verify --verbose
systemd-analyze verify /path/to/unit.service
podman quadlet --dryrun # 仅在当前版本提供该入口时使用
```

还包括读取文件的精确字段，例如 `nmcli connection show exam-static`、`grep -R` 检查 `repo` 或 `cron` 配置。静态通过不表示对象已经加载。

② [验证] 当前状态证据

当前证据回答“此刻是否生效”：

```
systemctl is-active httpd
systemctl is-enabled httpd
ss -ltn '( sport = :8082 )'
findmnt /srv/data
swapon --show
firewall-cmd --query-port=8082/tcp
getenforce
```

`is-active` 与 `is-enabled` 分属当前和持久配置，不得互相替代。

③ [验证] 功能证据

功能证据必须从实际消费者视角执行：

- 目标用户创建和读取协作文件；
- 从另一台主机请求 HTTP；
- 对新文件验证 ACL 继承；
- 访问 `autofs` 键触发远程挂载；
- 修改容器持久数据并重建容器后读取；
- 在精简环境下执行计划脚本并核对产物和日志。

功能验证应避免只由 `root` 执行，因为 `root` 可能绕过题目真正要求的访问控制。

④ [验证] 持久与重启证据

持久证据包括配置层和生命周期层：

```
nmcli -g connection.autoconnect connection show exam-static
systemctl is-enabled httpd
firewall-cmd --permanent --query-port=8082/tcp
findmnt --verify --verbose
loginctl show-user svcweb -p Linger
```

最终持久性通常仍需重启或重新登录验证。当前会话没有 `live VM`，因此本章只能给出推荐验证命令，不声称已执行。

⑤ [边界] 为证据写明证明边界

示例：

证据	能证明	不能证明
<code>httpd -t</code>	Apache 配置语法可解析	服务已运行、端口可访问
<code>ss -lnt</code>	本机进程监听端口	firewalld、SELinux、远端路由
<code>findmnt /srv/data</code>	当前已挂载	<code>/etc/fstab</code> 正确、重启后仍挂载
<code>podman ps</code>	当前用户容器运行	退出登录或重启后保持
<code>crontab -l</code>	调度条目存在	到点后成功执行

⑥ [验证] 定义完成状态

一项任务只有在所有必需证据层都关闭后才标记 `verified`。由于环境不可达而无法验证远端功能时，应标记 `blocked` 或 `deferred`，并记录缺失证据；不能把静态检查通过升级为完整成功。

[Cheatsheet] 静态看声明，当前看此刻，功能看消费者，持久看生命周期；每条证据都写清不能证明什么；缺证据就不写 PASS。

[操作专题] 用局部闭环、检查点和回归集合推进任务

综合任务的执行单位应是一个可验证的小闭环，而不是一口气修改多个子系统后再统一排错。局部闭环能把故障限制在最近的一组变更中，也便于记录恢复点。

① [操作] 每次只推进一个明确状态层

以 HTTP 服务为例：先让配置语法正确，再让服务运行，再确认监听，再处理 SELinux 和 firewalld，最后做客户端请求。若在配置语法尚未通过时同时修改防火墙，后续出现失败就会增加无关假设。

② [操作] 修改后立即运行最小验收

最小验收应贴近变更对象：

- 修改 `sudoers` 后立即 `visudo -cf`；
- 修改 `httpd` 后立即 `httpd -t`；
- 写入 `fstab` 后立即 `findmnt --verify`；
- 写入 `autofs map` 后先 `automount -m`；
- 写入 `Quadlet` 后重新加载用户 `manager` 并查询生成 `unit`。

③ [操作] 为共享对象建立回归集合

示例：修改 `/srv/web` 的挂载或标签后，回归集合至少包括：

```
findmnt -T /srv/web
→ stat/ls -Z
→ httpd 配置和进程
→ 本机请求
→ 远端请求
```

修改 `svcweb` 用户或 `home` 后，回归用户 `systemd`、容器存储和数据路径。

④ [操作] 保存检查点和差异

文本配置可保存权限和上下文：

```
cp -a /etc/httpd/conf.d/rhcsa33.conf \
/root/rhcsa33-backup/httpd-rhcsa33.conf.before
```

更重要的是记录变更前后的关键查询，例如 `nmcli connection show`、`semanage port -l`、`findmnt`。检查点不是为了堆积副本，而是为了明确最后一个可信状态。

⑤ [操作] 临时验证与最终持久配置分开

临时挂载、runtime 防火墙规则、手工启动容器可以用于快速验证，但必须在账本中标记为当前层证据；随后还要完成 `fstab`、`permanent firewalld` 或 `Quadlet` 等持久状态。临时成功若没有迁移到持久层，不能关账。

[Cheatsheet] 一次改一个层；修改后立即最小验收；共享对象变化触发回归；临时成功和持久配置分开记录。

[诊断专题] 卡题、误操作和交叉影响的证据式恢复

卡题时最危险的反应是扩大修改范围：服务不通就同时改配置、防火墙和 SELinux；挂载失败就重新格式化；网络不通就删除全部 profile。正确做法是固定症状和最后证据，明确哪一层已经证实，再选一条能区分假设的证据。

① [诊断] 保存症状和最后一条可信证据

记录具体对象、命令、错误信息和时间。不要只写“网络坏了”或“容器起不来”。例如：

```
症状：远端 curl 192.0.2.20:8082 超时
已证实：httpd -t 通过；ss 显示 0.0.0.0:8082 监听
未证实：active zone、runtime/permanent 端口、远端路由
```

② [诊断] 把假设限制在失败层

若本机 `curl` 成功而远端失败，应用内容和本机监听的优先级下降，下一步应看 `active zone`、`firewalld`、网络路径。若本机也失败，则先看服务、监听和 SELinux，不应直接调整远端防火墙。

③ [诊断] 选择最有区分度的下一条证据

高区分度证据应能一次排除多个假设。例如 `findmnt -T /srv/web` 能确认路径是否被其他文件系统覆盖；`ausearch -m AVC -ts recent` 能确认最近是否存在 SELinux 拒绝；`journalctl -u web33.service` 能把用户 unit 和容器运行错误关联起来。

④ [诊断] 只修复被证据支持的最小对象

看到 AVC 不等于关闭 SELinux；应确定目标文件类型、端口类型或 Boolean。看到 fstab 失败不等于注释全部条目；应定位具体行、设备标识和挂载点。

⑤ [诊断] 恢复后从失败层重新验证

修复 firewalld 后不必重新创建用户和 LVM，但应重新执行监听、runtime/permanent 查询和远端请求。恢复路径仍然遵守证据矩阵。

⑥ [诊断] 六个高频误判分支

症状	常见误判	下一条证据
新组关系不生效	<code>usermod</code> 失败	新登录会话中的 <code>id</code>
服务 active 但访问失败	重启次数不够	<code>ss</code> 、SELinux、firewalld、客户端
<code>lvs</code> 已增大但容量未变	LV 扩容失败	<code>findmnt</code> 、文件系统类型和 <code>df</code>
autofs 路径未出现在 <code>findmnt</code>	autofs 失败	先访问具体键，再查 <code>findmnt</code>
cron 脚本手工成功	调度已经正确	精简环境、身份、绝对路径和日志
root 的容器正常	rootless 任务完成	目标用户的 <code>podman</code> 与用户 manager

[Cheatsheet] 固定症状；区分已证实和未证实层；选高区分度证据；只修最小对象；从失败层重新验收。

[经典任务] 任务一：项目身份、协作目录与最小 sudo

环境与初始状态

- `opsadmins` 组和 `deploy` 用户尚不存在；
- `auditor` 用户已经存在；
- `/srv/project` 已存在，包含不可删除的现有文件和子目录；
- `httpd.service` 由另一任务维护；
- 本任务给出推荐操作与验收入口；实际结果需在 RHEL 9 环境填写。

目标终态

1. 创建组 `opsadmins`，GID 为 `3100`；

2. 创建用户 `deploy`，UID 为 `2201`，主组保持默认私有组，附加组为 `opsadmins`；
3. `/srv/project` 归属 `root:opsadmins`，目录模式为 `2770`；
4. 组成员能够在目录中协作，新建对象继承组；
5. `auditor` 对现有和未来对象具有读取文件、遍历目录的能力，但不能写入；
6. `deploy` 只能通过 `sudo` 执行 `/usr/bin/systemctl restart httpd.service`；
7. 不得删除或改写已有文件内容，不得授予通用 `root` Shell。

限制与验收

- 必须使用新登录会话验证附加组；
- 必须验证现有对象和新对象的 ACL；
- 必须使用 `visudo` 语法检查；
- 不能使用 `chmod 777`。

[参考解答] 任务一：身份、ACL 与 sudo 的证据闭环

① 调查与拆解

```
getent group opsadmins
getent passwd deploy auditor
stat -c '%n %U:%G %a %A' /srv/project
namei -om /srv/project
getfacl -R -p /srv/project
sudo -l -U deploy 2>/dev/null || true
```

若 `deploy` 或 `GID/UID` 已被占用，应停止并根据题目确认是否修改现有对象，不能重复创建或强制覆盖。

② 创建身份对象

```
groupadd -g 3100 opsadmins
useradd -u 2201 -G opsadmins deploy
getent group opsadmins
id deploy
```

`id deploy` 读取账号数据库，但当前已经打开的 `deploy` 会话不会自动刷新附加组。最终功能验证必须使用新的登录会话。

③ 设置协作目录的传统权限

```
chown root:opsadmins /srv/project
chmod 2770 /srv/project
stat -c '%n %U:%G %a %A' /srv/project
```

setgid 位使新建对象继承目录组；它不自动给既有子目录添加组写权限，也不能代替 ACL。

④ 为 auditor 设置现有与未来访问

对现有目录授予 `r-x`，对现有普通文件授予 `r--`：

```
find /srv/project -type d -exec setfacl -m u:auditor:r-x {} +
find /srv/project -type f -exec setfacl -m u:auditor:r-- {} +
```

对所有现有目录设置默认 ACL，使未来子对象继续继承：

```
find /srv/project -type d -exec setfacl -m d:u:auditor:r-x {} +
getfacl -p /srv/project
```

默认 ACL 仍受到创建程序请求模式和 ACL mask 影响。验证时应实际创建文件，而不是只读取默认条目。

⑤ 配置精确 sudo 规则

使用独立文件：

```
visudo -f /etc/sudoers.d/deploy-httpd
```

内容：

```
deploy ALL=(root) /usr/bin/systemctl restart httpd.service
```

校验：

```
chmod 0440 /etc/sudoers.d/deploy-httpd
visudo -cf /etc/sudoers
sudo -l -U deploy
```

不要加入通配符或 `/bin/sh`。精确命令行只允许目标 service 的 restart。

⑥ 以真实消费者身份验证

打开新的 deploy 登录会话后：

```
id
umask
touch /srv/project/deploy-test
stat -c '%n %U:%G %a %A' /srv/project/deploy-test
getfacl -p /srv/project/deploy-test
sudo /usr/bin/systemctl restart httpd.service
sudo -l
```

以 auditor 验证读取和拒绝写入：

```
runuser -u auditor -- cat /srv/project/deploy-test
runuser -u auditor -- sh -c 'echo denied >> /srv/project/deploy-test'
```

第二条预期被拒绝：不要在文档中伪造具体错误输出。

⑦ 证据矩阵

要求	静态/身份	当前	功能	持久
deploy 身份	getent、id deploy	新会话 id	能以组身份创建文件	账号数据库持久
协作目录	stat、getfacl	当前权限有效	deploy 创建对象并继承组	文件系统权限持久
auditor 只读	ACL 条目与 mask	当前访问	读取成功、写入被拒绝	默认 ACL 作用于新对象
最小 sudo	visudo -cf、 sudo -l -U	规则已加载	指定命令成功、其他命令被拒绝	sudoers 文件持久

典型错误

- 只执行 usermod -aG 后在旧会话中判断失败；
- 只给顶层目录 ACL，忽略已有子目录；
- 设置默认 ACL 后不创建新对象验证；
- sudoers 使用过宽通配符；
- 通过 root 读取文件代替 auditor 功能验证。

[经典任务] 任务二：静态网络、仓库与非标准 HTTP 服务

环境与初始状态

- 目标接口为 enp1s0；当前 DHCP profile 为 Wired connection 1；
- 通过控制台操作，允许保留旧 profile 作为临时恢复入口；
- repo.lab.example.com 和 client.lab.example.com 位于训练网络；
- /srv/web/index.html 已存在且内容不得改写；
- SELinux 必须保持 Enforcing。

目标终态

- 主机名：servera.lab.example.com；
- profile：exam-static；

- IPv4: 192.0.2.20/24 ; 网关 192.0.2.1 ; DNS 192.0.2.53 ;
- BaseOS: http://repo.lab.example.com/rhel9/BaseOS ;
- AppStream: http://repo.lab.example.com/rhel9/AppStream ;
- httpd 使用 /srv/web , 监听 8082/tcp ;
- 服务当前运行且开机启动 ;
- SELinux 文件与端口策略正确 ;
- firewalld runtime 与 permanent 均允许 8082/tcp ;
- client.lab.example.com 能读取首页。

[参考解答] 任务二：从网络 profile 到远端 HTTP 的完整访问链

① 调查并保留恢复入口

```
hostnamectl
nmcli -f NAME,UUID,TYPE,DEVICE,AUTOCONNECT connection show
nmcli device status
nmcli connection show 'Wired connection 1'
ip route
```

不要先删除当前 profile。确认控制台可用，并记录旧连接属性。

② 创建并核对静态 profile

```
nmcli connection add type ethernet ifname enp1s0 con-name exam-static \
  ipv4.method manual \
  ipv4.addresses 192.0.2.20/24 \
  ipv4.gateway 192.0.2.1 \
  ipv4.dns 192.0.2.53 \
  connection.autoconnect yes

nmcli connection show exam-static
hostnamectl set-hostname servera.lab.example.com
```

确认参数后激活：

```
nmcli connection up exam-static
ip -brief address show enp1s0
ip route
getent hosts repo.lab.example.com
```

新连接确认可用后，可避免旧 profile 抢占自动连接：

```
nmcli connection modify 'Wired connection 1' connection.autoconnect no
```

不需要为了完成任务删除旧 profile。

③ 配置并验证仓库

创建 `/etc/yum.repos.d/rhcsa33.repo` :

```
[rhel9-baseos]
name=RHEL 9 BaseOS training repository
baseurl=http://repo.lab.example.com/rhel9/BaseOS
enabled=1
gpgcheck=0

[rhel9-appstream]
name=RHEL 9 AppStream training repository
baseurl=http://repo.lab.example.com/rhel9/AppStream
enabled=1
gpgcheck=0
```

`gpgcheck=0` 只因为本训练任务明确提供无签名内网源；真实环境应遵循仓库签名策略。

```
dnf clean all
dnf repolist -v
dnf makecache --disablerepo='*' \
  --enablerepo=rhel9-baseos,rhel9-appstream
```

④ 安装工具并写入 httpd 配置

```
dnf install -y httpd policycoreutils-python-utils firewalld
```

创建 `/etc/httpd/conf.d/rhcsa33.conf` :

```
Listen 8082
<VirtualHost *:8082>
    DocumentRoot "/srv/web"
    <Directory "/srv/web">
        Require all granted
    </Directory>
</VirtualHost>
```

先验证静态配置：

```
httpd -t
```

⑤ 建立 SELinux 持久规则

先查询是否已有针对该路径的持久规则：

```
semanage fcontext -l | grep '/srv/web' || true
```

若没有匹配规则，再添加；若已有规则但类型错误，应在确认现有用途后使用 `-m` 修改：

```
semanage fcontext -a -t httpd_sys_content_t '/srv/web(/.*)?'\nrestorecon -RFv /srv/web\nmatchpathcon /srv/web /srv/web/index.html\nls -Zd /srv/web /srv/web/index.html
```

检查端口现状：

```
semanage port -l | grep '^http_port_t'
```

若 `8082/tcp` 尚未属于任何类型：

```
semanage port -a -t http_port_t -p tcp 8082
```

若端口已存在但类型错误，应在确认现有用途后使用 `-m` 修改；不要不经调查重复 `-a`。

⑥ 启动服务并检查监听

```
systemctl enable --now httpd.service\nsystemctl is-active httpd.service\nsystemctl is-enabled httpd.service\nss -lnt '( sport = :8082 )'\ncurl --fail http://127.0.0.1:8082/
```

本机请求通过后再处理外部访问层。

⑦ 配置 firewalld 双态

```
systemctl enable --now firewalld.service\nfirewall-cmd --get-active-zones\nZONE=$(firewall-cmd --get-zone-of-interface=enp1s0)\nprintf 'active zone for enp1s0: %s\\n' "$ZONE"
```

若输出为空或为 `no zone`，应先调查接口归属，不要把规则写入猜测的默认 `zone`。确认后在同一 `zone` 写入并查询 `runtime/permanent`：

```
firewall-cmd --zone="$ZONE" --add-port=8082/tcp
firewall-cmd --permanent --zone="$ZONE" --add-port=8082/tcp
firewall-cmd --zone="$ZONE" --query-port=8082/tcp
firewall-cmd --permanent --zone="$ZONE" --query-port=8082/tcp
```

⑧ 从远端消费者验证

在 `client.lab.example.com` :

```
curl --fail http://192.0.2.20:8082/
```

若本机成功而远端失败，优先调查 active zone、网络路径与远端路由，不要继续修改 DocumentRoot。

⑨ 证据矩阵

对象	静态	当前	功能	持久
网络	<code>nmcli con show exam-static</code>	地址、路由、DNS	仓库/NFS 名称可解析	autoconnect、重启后连接
仓库	repo 文件、 <code>repolist -v</code>	元数据可刷新	可查询或安装题目软件	repo 文件持久
httpd	<code>httpd -t</code>	active、监听 8082	本机与远端 curl	enabled
SELinux	fcontext/port 规则	Enforcing、标签生效	无策略拒绝且请求成功	<code>semanage</code> 规则持久
firewalld	permanent 查询	runtime 查询	远端访问	重载/重启后仍存在

典型错误

- 删除当前网络 profile 后失联；
- 只验证 `ping`，未验证 DNS 和仓库；
- 使用 `chcon` 代替持久 fcontext 规则；
- 只添加 runtime 防火墙规则；
- `httpd` active 但实际仍监听其他端口；
- 本机 curl 成功就跳过远端功能验收。

[经典任务] 任务三：从空闲设备建立 LVM、文件系统、Swap 与持久挂载

环境与初始状态

- 题目提供一块 2 GiB 训练磁盘 `/dev/vdb`；

- 设备可能含有旧签名，必须先调查；
- 系统盘及其他卷不得改动；
- 当前 `/srv/data` 不在其他挂载下。

目标终态

- GPT 分区表，使用 `/dev/vdb1` 作为 PV；
- VG: `vgexam`；
- LV: `lvdata`，大小 `1 GiB`，XFS，挂载 `/srv/data`；
- LV: `lvswap`，大小 `256 MiB`，启用为 Swap；
- 使用 UUID 写入 `/etc/fstab`；
- 当前挂载、写入和 Swap 正常；
- 配置通过启动前静态检查，重启后自动恢复。

[参考解答] 任务三：设备证据门与存储分层验收

① 通过设备证据门

```
lsblk -e7 -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid /dev/vdb /dev/vdb1 2>/dev/null || true
findmnt --source /dev/vdb 2>/dev/null || true
pvs --segments -o pv_name,vg_name,pv_size,pv_free
swapon --show
```

只有确认 `/dev/vdb` 是题目指定设备、没有被挂载、不是现有 PV/Swap，且旧签名不含需要保留的数据后，才能继续。若证据不清楚，应停止而不是运行 `wipefs -a`。

② 创建分区并等待内核识别

```
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart primary 1MiB 100%
partprobe /dev/vdb
udevadm settle
lsblk /dev/vdb
```


③ 创建 LVM 对象

```
pvccreate /dev/vdb1
vgcreate vgexam /dev/vdb1
lvcreate -L 1G -n lvdata vgexam
lvcreate -L 256M -n lvswap vgexam

pvs
vgs
lvs -o lv_name,vg_name,lv_size,lv_attr,devices
```

若 VG 空闲空间不足，应回到容量规划，不要使用强制选项或缩减其他未知 LV。

④ 创建文件系统和 Swap 签名

```
mkfs.xfs /dev/vgexam/lvdata
mkswap /dev/vgexam/lvswap
blkid /dev/vgexam/lvdata /dev/vgexam/lvswap
```

这些命令会写入数据结构，因此只能在对象身份已经确认后执行。

⑤ 建立当前挂载并验证写入

```
mkdir -p /srv/data
mount /dev/vgexam/lvdata /srv/data
findmnt /srv/data
touch /srv/data/.rhcsa33-write-test
stat /srv/data/.rhcsa33-write-test
```

确认 `/srv/data` 原来没有需要保留且被挂载遮蔽的内容。

⑥ 使用 UUID 写入 fstab

查询真实 UUID，不在讲义中编造，并在写入前检查空值与冲突：

```
DATA_UUID=$(blkid -s UUID -o value /dev/vgexam/lvdata)
SWAP_UUID=$(blkid -s UUID -o value /dev/vgexam/lvswap)
test -n "$DATA_UUID" && test -n "$SWAP_UUID"
grep -nE '(/srv/data|vgexam|lvswap)' /etc/fstab || true
grep -nF "UUID=$DATA_UUID" /etc/fstab || true
grep -nF "UUID=$SWAP_UUID" /etc/fstab || true
```

确认没有重复 UUID、设备或挂载点后再追加：

```
printf 'UUID=%s /srv/data xfs defaults 0 0\n' "$DATA_UUID" >> /etc/fstab
printf 'UUID=%s none swap defaults 0 0\n' "$SWAP_UUID" >> /etc/fstab
```

⑦ 验证 fstab 与 Swap

```
findmnt --verify --verbose
umount /srv/data
mount -a
findmnt /srv/data
swapon -a
swapon --show
```

可再次确认写入测试文件仍存在，证明挂载的是预期文件系统。

⑧ 重启门

重启前必须同时满足：

- `findmnt --verify` 没有阻断错误；
- UUID 与 `blkid` 一致；
- 挂载点存在；
- 当前 `mount -a` 和 `swapon -a` 成功；
- 系统其他 `fstab` 条目没有被误改；
- 具有控制台或恢复入口。

重启后推荐验证：

```
findmnt /srv/data
swapon --show
systemctl --failed
journalctl -b -p warning..alert --no-pager
```

证据矩阵

层	数据 LV	Swap LV
对象	<code>pvs/vgs/lvs</code>	<code>lvs</code>
签名	<code>blkid</code> 显示 XFS	<code>blkid</code> 显示 swap
当前	<code>findmnt /srv/data</code>	<code>swapon --show</code>
功能	实际写入与读取	内核已激活 Swap
持久	<code>fstab</code> UUID、 <code>findmnt --verify</code>	<code>fstab</code> UUID、重启后 <code>swapon</code>

典型错误

- 仅凭 `/dev/vdb` 名称判断空闲；
- 创建 LV 后忘记创建文件系统；

- fstab 使用错误 UUID 或重复挂载点；
- `mount -a` 未验证就重启；
- 把 `lvs` 容量当成 `df` 文件系统容量；
- 使用无调查的 `wipefs -a` 或 `--force`。

[经典任务] 任务四：autofs、时间同步、日志与计划任务

环境与目标

- 本地用户 `remote1` 已存在，home 为 `/rhome/remote1`；
- NFS 导出：`nfs.lab.example.com:/exports/home`；
- 使用 autofs 间接映射挂载到 `/rhome/<user>`；
- 唯一允许的上游时间源：`time.lab.example.com`；
- 每周三 15:30 由 root 执行 `/usr/local/sbin/rhcsa33-health`；
- 脚本写入 `/var/tmp/rhcsa33-health.txt`，并以 tag `rhcsa33-health` 写入 journal；
- autofs、chronyd、crond 在重启后自动启动。

限制

- 未访问具体键时不能仅因 `findmnt` 没有 NFS 挂载就判定 autofs 失败；
- 计划任务必须使用绝对路径和确定环境；
- `chronyd active` 不能替代指定时间源的选择与跟踪证据。

[参考解答] 任务四：远程依赖、时间可信链和可审计调度

① 检查前置依赖

```
getent hosts nfs.lab.example.com time.lab.example.com
id remote1
systemctl status autofs chronyd crond --no-pager
```

若名称解析失败，先回到“NetworkManager、IPv4 地址与路由”和“主机名、NSS 与 DNS 名称解析”章节的证据入口。

② 配置 autofs 间接映射

创建 `/etc/auto.master.d/rhome.autofs`：

```
/rhome /etc/auto.rhome
```

创建 `/etc/auto.rhome` :

```
* -fstype=nfs4,rw nfs.lab.example.com:/exports/home/&
```

设置合理权限后检查解析:

```
chmod 0644 /etc/auto.master.d/rhome.autofs /etc/auto.rhome
automount -m
systemctl enable --now autofs.service
systemctl restart autofs.service
```

③ 触发并验证具体键

```
ls -ld /rhome/remote1
findmnt -T /rhome/remote1
runuser -u remote1 -- test -r /rhome/remote1
```

autofs 是按访问触发; 未访问前没有远程挂载可以是正常状态。还应检查远端 UID/GID 与本地身份是否匹配。

④ 配置唯一 chrony 源

先保留配置副本并查看现有活动源:

```
cp -a /etc/chrony.conf /etc/chrony.conf.rhcsa33.before
grep -nE '^[[:space:]]*(server|pool)[[:space:]]' /etc/chrony.conf
```

本任务明确要求唯一源, 因此只注释活动的 `server / pool` 行, 再添加:

```
server time.lab.example.com iburst
```

重启并验证:

```
systemctl enable --now chronyd.service
systemctl restart chronyd.service
chronyc sources -v
chronyc tracking
timedatectl
```

必须关注源状态、当前选中源和 tracking, 而不是只看 service active。

⑤ 创建可审计脚本

`/usr/local/sbin/rhcsa33-health` :

```
#!/bin/bash
set -eu
PATH=/usr/sbin:/usr/bin:/sbin:/bin
stamp=$(date --iso-8601=seconds)
printf '%s host=%s status=ok\n' "$stamp" "$(hostname -f)" \
    >> /var/tmp/rhcsa33-health.txt
logger -t rhcsa33-health -- "health check completed at $stamp"
```

```
chmod 0755 /usr/local/sbin/rhcsa33-health
```

在接近 cron 的精简环境中手工验证：

```
env -i HOME=/root SHELL=/bin/bash PATH=/usr/sbin:/usr/bin:/sbin:/bin \
    /usr/local/sbin/rhcsa33-health
tail -n 2 /var/tmp/rhcsa33-health.txt
journalctl -t rhcsa33-health --since '-5 min'
```

⑥ 创建 cron 声明

/etc/cron.d/rhcsa33-health :

```
SHELL=/bin/bash
PATH=/usr/sbin:/usr/bin:/sbin:/bin
30 15 * * 3 root /usr/local/sbin/rhcsa33-health
```

```
chmod 0644 /etc/cron.d/rhcsa33-health
systemctl enable --now crond.service
systemctl is-active crond.service
systemctl is-enabled crond.service
```

当前会话无法等待真实触发时间，因此实际到点执行必须列入 live test；手工运行只证明脚本和环境，不证明 cron 已触发。

⑦ 证据矩阵

对象	静态	当前	功能	持久
autofs	automount -m	service active	访问键后 findmnt、目标用户读取	enabled、重启后再次触发
chrony	配置中的唯一源	service active	sources 选中、tracking 有效	enabled、重启后重新同步
cron	/etc/cron.d 字段	crond active	精简环境脚本、真实到点产物	enabled、文件持久

对象	静态	当前	功能	持久
日志	<code>logger</code> tag	journal 可查询	文件与日志时间关联	journal 持久策略取决于第 13 章配置

典型错误

- `autofs` 未触发就用 `findmnt` 判定失败；
- `map` 中通配符和 `&` 对应错误；
- `chronyd` active 但指定源为 `^?`；
- `cron` 文件漏写用户字段；
- 使用相对路径或依赖交互 Shell 环境；
- 手工运行成功就宣称计划任务已执行。

[经典任务] 任务五：rootless Podman、用户 systemd、Quadlet 与持久数据

环境与目标

- 用户 `svcweb` 已存在；
- 镜像：`registry.lab.example.com/rhel9/httpd-24:latest`，容器内监听 `8080`；
- 容器名：`web33`；主机端口：`8088`；
- 数据目录：`/home/svcweb/site`，包含 `index.html`；
- 使用 Quadlet 文件 `web33.container`；
- SELinux 保持 Enforcing；
- `svcweb` 退出登录后用户服务仍运行；
- 宿主机重启后容器自动恢复，数据保持。

限制

- 所有 Podman 与 `systemctl --user` 操作必须在 `svcweb` 用户上下文执行；
- 不得用 `root` 容器代替 `rootless` 容器；
- 不得把手工容器成功扩大为 Quadlet 持久状态正确。

[参考解答] 任务五：从手工容器到用户级持久生命周期

① 确认用户上下文和路径

以 `svcweb` 的真实登录会话执行：

```
id
printf '%s\n' "$HOME" "$XDG_RUNTIME_DIR"
podman info
mkdir -p "$HOME/site" "$HOME/.config/containers/systemd"
stat -c '%n %U:%G %a' "$HOME/site"
```

若 `podman info` 显示 `rootful` 存储路径或 `HOME` 不属于 `svcweb`，应先修正会话上下文。

② 先用手工容器验证镜像、端口和数据

```
podman pull registry.lab.example.com/rhel9/httpd-24:latest
podman run --rm --name web33-test \
  -p 8088:8080 \
  -v "$HOME/site:/var/www/html:Z" \
  registry.lab.example.com/rhel9/httpd-24:latest
```

在另一个会话验证：

```
curl --fail http://127.0.0.1:8088/
ls -Zd /home/svcweb/site
```

手工验证通过后停止测试容器。若失败，先解决镜像、端口、数据或 SELinux，不要直接进入 Quadlet。

③ 写入 Quadlet 声明

`/home/svcweb/.config/containers/systemd/web33.container :`

```
[Unit]
Description=RHCSA 33 rootless web container
After=network-online.target
Wants=network-online.target

[Container]
Image=registry.lab.example.com/rhel9/httpd-24:latest
ContainerName=web33
PublishPort=8088:8080
Volume=/home/svcweb/site:/var/www/html:Z

[Service]
Restart=always

[Install]
WantedBy=default.target
```

④ 重新生成并检查用户 unit

仍在 `svcweb` 会话：

```
systemctl --user daemon-reload
systemctl --user list-unit-files | grep web33
systemctl --user start web33.service
systemctl --user status web33.service --no-pager
podman ps
curl --fail http://127.0.0.1:8088/
```

Quadlet 生成的 unit 在不同版本中可能显示为 `generated`，不应机械要求 `systemctl --user enable`。
[Install] WantedBy=default.target 承担启动关联，验证应结合 `list-unit-files`、default target 依赖和重启结果。

⑤ 启用 linger

由 root 执行：

```
loginctl enable-linger svcweb
loginctl show-user svcweb -p Linger -p State
```

退出 `svcweb` 登录后，从 root 或其他用户验证端口和进程仍存在：

```
curl --fail http://127.0.0.1:8088/
loginctl user-status svcweb
```

⑥ 验证持久数据

修改 host 数据文件后确认容器读取变化；重启容器或 unit 后再次确认。不要把容器可重建与数据持久化混为一谈。

⑦ 重启后验证计划

```
loginctl show-user svcweb -p Linger
curl --fail http://127.0.0.1:8088/
# 登录为 svcweb 后
systemctl --user status web33.service --no-pager
podman ps
```

当前会话没有 live VM，因此这些命令是推荐验收入口，不是已执行结果。

证据矩阵

层	证据
用户上下文	<code>id</code> 、HOME、rootless <code>podman info</code>
容器功能	手工容器、端口、curl、数据目录标签

层	证据
Quadlet 静态	<code>.container</code> 文件、生成 unit 可见
当前	用户 unit active、 <code>podman ps</code> 、curl
退出登录	linger、退出后 curl 与 user-status
重启	开机后 curl、用户 unit、数据仍在

典型错误

- root 执行 `podman`，验证了错误的容器存储；
- 直接写 Quadlet，未先隔离镜像或数据问题；
- 对生成 unit 机械执行 enable 并把失败当成 Quadlet 故障；
- 漏写 `:Z` 导致 SELinux 拒绝；
- 只验证容器运行，未验证数据和退出登录生命周期；
- 启用 linger 后不做退出登录测试。

[经典任务] 任务六：模拟考试级最终检查、重启决策与恢复演练

一台训练主机已经执行了多项操作，但存在以下不完整状态：

1. `deploy` 已加入 `opsadmins`，当前打开的旧会话仍看不到该组；
2. `exam-static` 当前激活，但 `connection.autoconnect=no`；
3. 两个 repo 文件均存在，其中 AppStream 元数据刷新失败；
4. httpd active，配置端口为 `8082`，但只有 runtime firewalld 规则，SELinux 端口类型未知；
5. `vgexam/lvdata` 已创建并挂载，fstab 的挂载点误写为 `/srv/date`；
6. autofs master map 正确，indirect map 中远端路径拼写错误；
7. 健康检查脚本手工运行成功，`/etc/cron.d` 条目漏写用户字段；
8. `svcweb` 手工容器成功，Quadlet 的数据路径写成 `/home/svcweb/sites`，linger 未启用。

要求：

- 不删除现有对象重做；
- 建立调查基线、依赖图和风险队列；
- 按最小修改修复；
- 逐项关闭静态、当前、功能和持久证据；
- 对共享对象执行回归；
- 判断是否允许重启；
- 若启动失败，指出应进入的恢复路径；
- 重启后形成最终证据矩阵。

[参考解答] 任务六：从缺陷清单到最终可交卷状态

① 先建立缺陷账本，而不是立即修复

ID	对象	已知症状	风险	下一条证据
U1	deploy 会话	旧会话无附加组	R1	新登录 <code>id</code>
N1	exam-static	当前 active、autoconnect no	R3	<code>nmcli con show</code>
R1	AppStream	元数据失败	R1	DNS、HTTP、baseurl
W1	HTTP 访问链	active 但安全层不完整	R2	<code>ss</code> 、semanage、双态 firewalld
S1	fstab	挂载点拼写错误	R4	<code>findmnt --verify</code> 、当前挂载
A1	autofs map	远端路径错误	R2	<code>automount -m</code> 、访问键、journal
C1	cron	漏用户字段	R1	文件结构、crond 日志
P1	Quadlet	路径和 linger 错误	R2	用户 unit journal、路径、loginctl

② 建立依赖与执行批次

推荐批次：

批次 1：只读基线与身份新会话验证
批次 2：网络 autoconnect、DNS、仓库
批次 3：httpd 配置/监听、SELinux、firewalld、远端请求
批次 4：修复 fstab 并通过重启证据门
批次 5：autofs、chrony、cron 的功能与日志
批次 6：svcweb Quadlet、数据、linger
批次 7：共享对象回归与最终矩阵
批次 8：满足重启后重启验收

网络先于仓库、NFS 和镜像；fstab 在最终重启前必须优先关闭；用户和路径修复后需要回归容器和权限。

③ 分项最小修复

- U1：不重复 `usermod`，打开新的 deploy 登录会话并执行 `id`。若新会话仍缺组，再查询 `/etc/group` 或 NSS。
- N1： `nmcli connection modify exam-static connection.autoconnect yes`，核对 profile，不需要重新创建。
- R1：对失败 repo 使用 `dnf repoinfo`、`curl` 或元数据刷新区分 DNS、路径和 HTTP；只修正错误的 baseurl。
- W1：保留现有服务，按 `httpd -t → ss → semanage port → firewalld runtime/permanent → 本机/远端 curl 闭环`。

- S1: 把 `/srv/date` 修正为 `/srv/data`，执行 `findmnt --verify`、卸载/`mount -a` 回归；禁止通过注释整份 `fstab` 绕过。
- A1: 修正 indirect map 的导出路径，`automount -m` 后重启 `autofs`，访问具体键并查 `journal`。
- C1: 在 `/etc/cron.d` 时间字段后补 `root`，保持绝对路径和显式 `PATH`；手工精简环境验证与真实到点验证分开记录。
- P1: 在 `svcweb` 会话修正 `Volume` 路径，重新加载用户 `manager`，检查 `journal`，启用 `linger` 后退出登录验证。

④ 共享对象回归

- 网络变化后：仓库、NFS 名称解析、HTTP 远端访问、镜像仓库；
- `/srv/data` 与 `fstab` 修复后：依赖该路径的服务和文件所有权；
- `svcweb` 路径修复后：SELinux 标签、容器数据和 `Quadlet`；
- 时间或时区变化后：日志事件与计划任务时间解释。

⑤ 重启门判定

只有以下条件全部满足才允许重启：

```
NetworkManager profile 持久且可达
→ findmnt --verify 无阻断错误
→ 当前 mount -a / swapon -a 正常
→ 关键服务 enabled 且当前功能已验证
→ firewalld permanent 与 SELinux 持久规则正确
→ autofs / cron / chrony 声明可解析
→ linger 与 Quadlet 启动关系已建立
→ 控制台或恢复入口可用
```

若 `fstab` 仍有错误、网络仅当前可用或用户容器尚未建立持久关系，应推迟重启并把任务标记 `blocked`。

⑥ 重启失败的恢复入口

- `fstab` 或挂载导致紧急模式：进入“启动链、GRUB、Target 与系统恢复”，在救援环境定位并修复具体 `fstab` 行；
- 网络失联：使用虚拟机控制台，检查 `profile` 和设备绑定；
- 用户容器未启动但系统正常：先保持系统在线，登录目标用户检查用户 `unit journal`；
- 服务访问失败：不要进入系统恢复，回到服务访问链逐层诊断。

⑦ 重启后最终矩阵

deploy 组关系静态：NSS 记录当前：新会话 `id` 功能：协作目录操作重启/新会话：关系仍正确结果：
`verified` / `pending`

网络与仓库静态：`profile` 与 `repo` 定义当前：地址、DNS、元数据功能：访问依赖资源重启后：自动连接且元数据可用结果：`verified` / `pending`

HTTP 完整访问链静态：应用、SELinux 与永久防火墙当前：服务 active 且监听功能：远端 curl 重启后：自动启动并可访问结果：verified / pending

存储静态：fstab 中稳定标识与路径当前：文件系统挂载、Swap 激活功能：写入与读取重启后：自动挂载与激活结果：verified / pending

autofs、cron 与时间静态：map、cron 与 chrony 声明当前：相关服务运行功能：触发、产物与日志重启后：能够再次触发结果：verified / pending

rootless 容器静态：Quadlet 与 linger当前：用户 unit 与容器功能：HTTP 与持久数据重启后：无登录自动恢复结果：verified / pending

当前材料只能提供静态设计，因此实际结果栏必须由 RHEL 9 live test 填写，不能预先写成全部 verified。

⑧ 最终交卷检查

从原始题目逐条读回，而不是从命令历史读回：

- 1. 每个原题都有矩阵行；
- 2. 每行的参数与题目完全一致；
- 3. 限制条件没有被捷径破坏；
- 4. 当前、功能和持久证据没有互相代替；
- 5. blocked 项明确列出缺失证据；
- 6. 重启后只复查矩阵，不凭记忆判断。

[本章收束] 一致终态比命令数量更重要

RHCSA 综合任务的稳定能力不是更快地背出命令，而是能够把模糊要求转换为对象状态，知道每条证据能证明什么，在共享对象变化后回归相关功能，并在重启前主动关闭启动风险。

最终工作流可以压缩为：

- 读原题并拆评分行
 - 调查对象身份和初始状态
 - 建立依赖图、共享对象和风险队列
 - 按局部闭环完成最小变更
 - 记录静态、当前、功能和持久证据
 - 修改共享对象后回归
 - 通过重启门
 - 重启后按原题与证据矩阵逐项验收

主要判断表

看到的证据	可以得出的结论	仍不能得出的结论
命令退出码为 0	本次调用按命令自身约定成功	目标系统已经达到完整终态
配置语法检查通过	声明可被对应程序解析	服务已运行、端口可达、功能正确

看到的证据	可以得出的结论	仍不能得出的结论
<code>active</code>	当前 unit 正在运行	已 enabled、客户端可用、重启后保持
当前挂载存在	当前路径已经挂载	fstab 正确、重启后仍挂载
runtime 防火墙已放行	当前规则集允许该服务或端口	permanent 状态已保存
本机请求成功	应用、监听和本机路径大体成立	远端路由、zone 与外部访问成立
root 容器运行	root 的容器实例可用	目标用户的 rootless 生命周期正确
重启后仍成立	持久链在本次重启中生效	其他尚未验收的任务也正确

工作方法

在考试中，用任务账本限制遗漏，用依赖图减少返工，用证据矩阵避免把局部成功扩大为完成。在真实维护窗口中，同样的方法可以转化为变更前基线、风险门、回退条件、消费者验证和变更后证据。

向后续学习与工作交接

本章是 RHCSA 手工系统管理的收束章，没有下一章需要继续展开某个 RHCSA 子系统。后续进入 RHCE 或实际自动化时，应保留本章的对象模型、依赖关系和验收矩阵，再把“手工修改接口”替换为模块、变量、模板与幂等任务。自动化只改变实现方式，不降低终态证据标准。